第 8 章練習解答　主記憶體

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS08-E01　Paging 轉換

1. 4 KiB=2^12，offset = 0x2A7C & 0xFFF = 0xA7C，page number = 0x2。physical base = 9×0x1000=0x9000，physical address=0x9A7C。
2. MMU 的 read-only 權限檢查失敗，產生 page protection fault；核心不得先完成寫入再回報。
3. 不一定。無效表示目前不在有效映射，可能是 demand paging 尚未載入、頁面曾被換出，或確實是錯誤位址；需查 VMA/資源規則。

檢查。 每一步保留位元寬度與權限判斷。

OS08-E02　有效存取時間

1. 命中成本 2+100=102 ns；miss 成本 2+100+100=202 ns，其中一次讀 page-table entry，一次存取目標資料，不能把 frame number 再算成獨立 memory read。EMAT=0.95×102+0.05×202=107 ns。
2. EMAT=0.8×102+0.2×202=122 ns。
3. 需加入 page-fault rate f 與 fault service time F，例如 `(1-f)×EMAT_no_fault+f×F`，並確保 fault cost 與正常存取不重複計算。

檢查。 先寫出哪些成本在 hit 與 miss 路徑發生。

OS08-E03　Page 大小取捨

1. 33 KiB 需 9 頁（36 KiB），在題設獨立 page-aligned 配置下每物件尾端恰浪費 3 KiB，100 個合計 300 KiB；若允許物件共用頁或打包，此結論不適用。
2. 同樣 virtual range 使用較大 page 時需要較少 entries，可降低 table 層級與 TLB footprint。
3. 不一定。小 page 減少單一物件尾端浪費，但 page table、TLB miss 與 fault 處理更多，可能增加總足跡。需以 workload 實測。

檢查。 答案同時保留空間與轉換成本。

第 8 章練習解答　主記憶體 （續）

OS08-E04　陌生 x86-64 位址的完整 walk

1. bits63:48=0、bit47=0，因此 canonical。indices 為 0x024/0x0D1/0x0B3/0x189，offset=0xABC。
2. entry offsets 為 index×8：0x120/0x688/0x598/0xC48；addresses 為 0x7120/0x8688/0x9598/0xAC48。entry values 可為 `0x8007`、`0x9007`、`0xA007`、`0x12AB007`，表示 next bases 與 P/RW/US。
3. physical address=`0x12AB000+0xABC=0x12ABABC`。
4. PTE 末三位從 7 改為 5，表示 present 與 user 仍成立但 RW=0。read 成功且仍到 0x12ABABC；write 產生 protection fault，不能先寫後報錯。

檢查。 每一步同時保留 index、entry address、entry value 與 offset，不能只給最後的 PA。

OS08-E05　canonical、present 與權限的分流

1. A 的 bits63:48=0、bit47=0，因此 canonical；indices=0x048/0x1A2/0x167/0x001，offset=0x234。
2. D 的 bits63:48=0、bit47=1，noncanonical，在 table walk 前拒絕。B 的 PML4E present=0，遇到該層即可 page fault，不能讀下層。
3. B 的 read/write/instruction fetch都不能完成；C 的 mapping 可讀寫但 NX 只阻止 instruction fetch，資料 read/write仍可依 RW 成功。D 三個操作都在 walk 前拒。
4. 不能。present 只證明翻譯路徑可使用；RW 與 NX 是隨後的操作權限。最少要逐列 canonical、present、U/S、RW、NX 與 operation result，並用模型回歸所有分支。

檢查。 canonical 與 permission 是串聯條件；案例必須在正確的 fault stage 停下。

第 8 章練習解答　主記憶體 （續）

OS08-E06　ARM table layout、domain 與權限

1. N=0 的 root 需 4096×4=16KiB、16KiB 對齊：14000–17FFF 與 18000–1BFFF 合法且不重疊。L2 需256×4=1KiB、1KiB 對齊：1C000–1C3FF 與1C400–1C7FF 合法且不重疊。P1 indices=(4,A6)、entries 14010/1C298；P2 indices=(7,C8)、entries 1801C/1C720。
2. L1 bits[1:0]=01為coarse；base分別1C000/1C400，domain=(bits8:5)=3。L2 frame=(bits31:12)=54000；AP2=(bit9)=0、AP1=(bit5)=1、AP0=(bit4)=0，故AP=010；XN=0、nG=1、TEX=001、C=B=0。D3=Client時PL0 read成功、write permission fault。
3. P1 的 L1/L2 entry absent 後，P1 read 是 translation fault；P2 的獨立表仍 valid，P2 read 成功。refcount 由 2 降為 1，還不能回收 frame。
4. D3=Manager（11）時short-descriptor不檢查AP，相同descriptor不能保證write fault。AFE=1時AP[0]變成Access flag，只剩AP[2:1]參與權限模型，必須重新解碼，也不能沿用AP=010的fault表。若把raw誤寫成0x54802，正確decode是AP000，PL0 read即permission fault；0x54022雖AP010，但TEX/C/B=0/0/0為Strongly-ordered，不能憑XN=0宣稱instruction fetch。

檢查。 先驗 storage size/alignment/non-overlap，再解 raw words；domain、AFE 與 TLB 條件改變時不得沿用舊結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 8 章練習解答　主記憶體 （續）

原創驗算程式附錄: address_translation.py

```
"""Read-only page-translation model with mapping and permission checks.

Missing entries are rejected; demand paging and segmentation are not modeled.

Run:
    python address_translation.py
"""

PAGE_SIZE = 4096
PAGE_MASK = PAGE_SIZE - 1


def translate(virtual_address: int, table: dict[int, tuple[int, str]]) -> int:
    if virtual_address < 0:
        raise ValueError("virtual address must be non-negative")
    page = virtual_address >> 12
    offset = virtual_address & PAGE_MASK
    entry = table.get(page)
    if entry is None:
        raise PermissionError("invalid page")
    frame, permission = entry
    if "r" not in permission:
        raise PermissionError("read denied")
    return frame * PAGE_SIZE + offset


def main() -> None:
    table = {2: (9, "r")}
    assert translate(0x2A7C, table) == 0x9A7C
    try:
        translate(0x3A7C, table)
    except PermissionError:
        pass
    else:
        raise AssertionError("invalid page should fail")
    print("0x2A7C -> 0x9A7C; invalid page rejected")


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study